8. AI large model online voice assistant

Note: The Raspberry Pi 5 1GB version does not have this package and cannot run this case.

1. Online Voice Configuration

Before setting up automatic startup, we must ensure that the program itself can operate independently while online. This can be accomplished by modifying the configuration file.

First, enter the Docker container by typing the following command in the terminal:

```
./ros2_docker.sh
```

If you need to enter other commands in the same Docker container later, simply enter ./ros2_docker.sh again in the host terminal.

1. **Open the configuration file**:

   ```
   vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
   ```

2. **Modify Configuration Parameters**:
   Please check the following parameters in the file and ensure that their values match those shown below. If the parameters do not exist, add them.

   ```
   asr:                                        #Voice node parameters
     ros__parameters:
       VAD_MODE: 2                             #vad sensitivity
       sample_rate: 16000                      #asr recording audio sampling rate
       frame_duration_ms:  30                  #vad frame size in ms
       use_oline_asr: True                    #Whether to use online ASR recognition
   (True uses online, False uses offline)
       mic_serial_port: "/dev/ttyUSB0"         #Microphone serial port alias
       mic_index: 0                            #Microphone Index
       language: 'zh'                          #asr language

   model_service:                              #Model server node parameters
     ros__parameters:
       language: 'zh'                          #Large Model Interface Language
       useolinetts: True                       #Whether to use online speech
   synthesis (True uses online, False uses offline)
       llm_platform: 'tongyi'                  # Here we take Tongyi as an example
   ```

3. `use_oline_asr` and `useolinetts` must be set to `tongyi`

- `use_oline_asr` and `useolinetts` must be set to `tongyi`.
- `llm_platform` must be set to `tongyi`.

  With this setup, everything is online.

3. **Save the file** and **recompile** the project to apply the changes. This requires building within a Docker container:

```
cd ~/yahboom_ws
colcon build
source install/setup.bash
```

After completing this step, the program is now a fully online voice service.

2. Create a Systemd Service

Now, we will create a systemd service to automatically run `largemodel_control.launch.py` at system startup.

Note: If you are using the Yabo AI Large Model factory image, these scripts are already created; you only need to restart the auto-start container.

2.1 Create a Docker Container Startup Script

1. **Create a Script File**:
   In your home directory (~), create a file named `ros2_docker_autostart.sh`.

```
vim ~/ros2_docker_autostart.sh
```

2. **Write the Script Content**:
   Copy and paste the following content into the script file.

```
#!/usr/bin/env bash

set -e

CN=${CONTAINER_NAME:-ros_humble_ai_service}
IMG=${IMAGE_NAME:-ros_humble_ai:v1.0}
WS=${HOST_WS_DIR:-"$HOME/yahboom_ws"}
ROS=${ROS_DISTRO:-humble}
CMD=${LAUNCH_CMD:-"ros2 launch largemodel largemodel_control.launch.py"}
AUID=${AUDIO_UID:-$(id -u)}
XAUTH=${XAUTHORITY:-"$HOME/.Xauthority"}

mkdir -p "$WS"

x=()
if [ -d /tmp/.X11-unix ]; then
  command -v xhost >/dev/null 2>&1 && xhost +local:root || true
  x+=( -e DISPLAY=${DISPLAY:-:0} -e QT_X11_NO_MITSHM=1 -v /tmp/.X11-unix:/tmp/.X11-unix )
  [ -f "$XAUTH" ] && x+=( -v "$XAUTH:/root/.Xauthority" -e XAUTHORITY=/root/.Xauthority )
fi

a=()
```

```
if [ -S "/run/user/$AUID/pulse/native" ]; then
  a+=( -e PULSE_SERVER=unix:/run/user/$AUID/pulse/native -v
"/run/user/$AUID/pulse:/run/user/$AUID/pulse:ro" )
  [ -d "$HOME/.config/pulse" ] && a+=( -v
"$HOME/.config/pulse:/root/.config/pulse:ro" )
fi

d=()
[ -e /dev/video0 ] && d+=( --device=/dev/video0 )
[ -d /dev/bus/usb ] && d+=( --device=/dev/bus/usb )
[ -d /dev/snd ] && d+=( --device=/dev/snd )
[ -e /dev/mic ] && d+=( -v /dev/mic:/dev/mic )

if docker ps -a --format '{{.Names}}' | grep -qx "$CN"; then
  docker ps --format '{{.Names}}' | grep -qx "$CN" || docker start "$CN"
>/dev/null
else
  docker run -d \
    --name "$CN" --net=host --privileged --restart unless-stopped \
    --security-opt apparmor:unconfined \
    -v "$WS:/root/yahboom_ws:rw" \
    "${d[@]}" "${x[@]}" "${a[@]}" \
    "$IMG" \
    bash -lc "source /opt/ros/$ROS/setup.bash; [ -f
/root/yahboom_ws/install/setup.bash ] && source
/root/yahboom_ws/install/setup.bash; $CMD"
fi

echo "OK: $CN"
```

3. **Save and exit**.
4. **Confirm that Docker starts at boot**:

```
sudo systemctl enable --now docker
```

5. **Give the script execution permissions** and run:

```
chmod +x ~/ros2_docker_autostart.sh
./ros2_docker_autostart.sh
```

6. **Verify**:

```
docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' ros_humble_ai_service
```

The output should show "unless-stopped". From now on, Docker will automatically start the container at each boot, enabling the voice assistant to automatically start.

2.2 If you are using the YABO AI Large Model factory image, you do not need to recreate the script. Simply run the following command to restart the autostart service:

```
~/ros2_docker_autostart.sh
```

2.3 Related Commands:

Stop the container immediately, which will also stop the autostart service:

```
docker stop ros_humble_ai_service
```

View the Docker container's running logs:

```
docker logs -f ros_humble_ai_service
```

3. CSI Camera Autostart Service

To access the CSI camera feed in Docker, you need to start host_stream.py on the host machine. Therefore, if you need to access the CSI camera feed during autostart, you must also add this file to the Systemd startup service.

Note: If you are using the YABO AI Large Model factory image, these scripts are already created; you only need to restart the autostart service.

1. Enter the command to write to the file.

   ```
   sudo vim /etc/systemd/system/host-stream.service
   ```

2. Copy the following content into the file.

   ```
   [Unit]
   Description=Host stream (runs ~/host_stream.py)
   After=network.target

   [Service]
   Type=simple
   User=pi
   WorkingDirectory=/home/pi
   ExecStart=/usr/bin/python3 /home/pi/host_stream.py
   Restart=on-failure
   RestartSec=5
   StandardOutput=append:/home/pi/host_stream.log
   StandardError=append:/home/pi/host_stream.log

   [Install]
   WantedBy=multi-user.target
   ```

3. Save and exit.

4. Apply and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable host-stream.service
sudo systemctl start host-stream.service
```

5. Check the service status:

```
sudo systemctl status host-stream.service

If you see `Active: active (running)`, the service has started
successfully!
```

6. If you no longer need the CSI camera's automatic startup service, run the following command to stop it:

```
sudo systemctl stop host-stream.service
sudo systemctl disable host-stream.service
```

If you are using the factory image of Yabo's AI large model, you do not need to recreate the script. Just run the following command to restart the auto-start service:

```
sudo systemctl enable host-stream.service
sudo systemctl start host-stream.service
```